

POC Regression Report

Thai Medical OCR Pipeline Blocks

NLP_RESEARCH_TOPICS Workspace

2026-04-06

1 Executive Summary

This report summarizes the end-to-end regression run of six Proof-of-Concept (POC) modules used in the Thai medical OCR research pipeline. All modules are currently in a passing state after targeted bug fixes. The final state is:

- 6/6 modules pass runnable tests.
- 4 modules required code fixes during regression.
- 1 module (PaddleOCR v5) required a backend migration from Paddle runtime to HuggingFace transformers.

2 Environment

- OS: Linux
- GPU: NVIDIA GeForce RTX 5070 (Blackwell, SM_120)
- CUDA target: cu128 (required for SM_120 compatibility)
- Python/package manager: uv
- Workspace root: /home/puem/Desktop/NLP_RESEARCH_TOPICS

3 Pipeline Context

The tested modules map to the Thai medical OCR pipeline as follows:

- Stage 2 (Layout Detection): DocLayout-YOLO
- Stage 3 (Text Recognition): EasyOCR Thai, PaddleOCR v5, GOT-OCR2
- Stage 4 (Post-correction): ByT5 OCR correction, WangchanBERTa correction

4 Test Inputs

Two main input classes were used:

1. Synthetic samples generated by each module's `generate_sample.py`
2. A real academic paper image: `/home/puem/Desktop/NLP_RESEARCH_TOPICS/test_image.png` (single-column page)

5 Final Regression Results

Module	Stage	Status	Key Result
DocLayout-YOLO	Stage 2	PASS	Detected layout regions successfully on synthetic and real paper images.
EasyOCR Thai	Stage 3	PASS	42 regions (synthetic), 107 regions (real paper). JSON export stable after dtype fix.
PaddleOCR v5 (transformers backend)	Stage 3	PASS	8 regions (synthetic), 53 regions (real paper). Detection + recognition pipeline executes on GPU.
GOT-OCR2	Stage 3	PASS	53 lines, 2978 characters on real paper in about 6.9 seconds.
ByT5 OCR correction	Stage 4	PASS	8 sample corrections executed; expected weak exact-match because model is not fine-tuned for this domain.
WangchanBERTa correction	Stage 4	PASS	8 sample mask-fills executed; 1/8 exact match on quick sanity set.

6 Issue Log and Fixes

1) EasyOCR Thai JSON serialization failure

Symptom: `TypeError: Object of type int32 is not JSON serializable`

Root cause: Bounding box values remained NumPy scalar types.

Fix: Cast bbox coordinates to Python floats before JSON write.

2) GOT-OCR2 API incompatibility

Symptom: Missing `model.chat()` method in current transformers implementation.

Root cause: Runner assumed an older API style.

Fix: Switched to `AutoModelForImageTextToText` + `AutoProcessor` and generation-based inference.

3) Common CUDA device property bug

Symptom: Access to `total_mem` failed.

Root cause: Incorrect PyTorch CUDA property name.

Fix: Replaced with `total_memory` in affected runners.

4) WangchanBERTa tokenizer dependency failure

Symptom: Tokenizer initialization errors referencing `tiktoken`/SentencePiece extractor requirements.

Root cause: Missing protobuf runtime dependency in environment.

Fix: Added `protobuf` to project dependencies.

5) PaddleOCR v5 runtime failure and migration

Symptom: Paddle runtime failed with low-level CPU path error (OneDNN / PIR conversion failure).

Root cause: Incompatibility in the tested Paddle stack for this environment.

Fix: Migrated module to HuggingFace transformers PP-OCRv5 models:

- Detection model: PaddlePaddle/PP-OCRv5_server_det_safetensors
- Recognition model: PaddlePaddle/PP-OCRv5_server_rec_safetensors

Additionally repaired OpenCV wheel state and adapted detection parsing from polygon format (N,4,2) to axis-aligned crop boxes for recognition.

7 Thai-Language Caveat

The current PaddleOCR v5 transformers path uses the server recognition model with a non-Thai character inventory for the tested checkpoint. This is sufficient for English-heavy pages (as shown on the real paper test), but Thai-heavy medical forms may degrade in character fidelity. For production Thai documents, a Thai-specific recognition checkpoint (with compatible preprocessor metadata) is recommended.

8 Reproducibility Commands

From workspace root:

```
cd poc/doclayout-yolo && bash run.sh
cd ../easyocr-thai && bash run.sh
cd ../paddleocr-v5 && bash run.sh
cd ../got-ocr2 && bash run.sh
cd ../byt5-ocr-correction && bash run.sh
cd ../wangchanberta-correction && bash run.sh
```

Real-image spot checks:

```
cd poc/paddleocr-v5
uv run python poc_runner.py --image ../../test_image.png --save-json
```

9 Conclusion

The regression objective is met: all six modules now run successfully in the current environment. The highest-risk blocker (Paddle runtime instability) has been removed through a tested migration path, and the workspace is now in a stable state for next-stage integration and comparative evaluation experiments.